

# Statistics Interview Questions & Answers Cheat Sheet

## Table of Contents

- 1. [Descriptive Statistics](#)
- 2. [Probability Theory](#)
- 3. [Probability Distributions](#)
- 4. [Sampling and Central Limit Theorem](#)
- 5. [Hypothesis Testing](#)
- 6. [Confidence Intervals](#)
- 7. [Correlation and Regression](#)
- 8. [ANOVA and Chi-Square Tests](#)
- 9. [Bayesian Statistics](#)
- 10. [Time Series Analysis](#)
- 11. [Experimental Design](#)
- 12. [Statistical Power and Effect Size](#)

## Descriptive Statistics

**Q1: What are the measures of central tendency and when to use each?**

**Answer:**

**Measures of Central Tendency:**

| Measure | Formula                        | Best Use Case        | Pros                     | Cons                      |
|---------|--------------------------------|----------------------|--------------------------|---------------------------|
| Mean    | $\bar{x} = \frac{\sum x_i}{n}$ | Normal distributions | Mathematical properties  | Sensitive to outliers     |
| Median  | Middle value when sorted       | Skewed distributions | Robust to outliers       | Less mathematical utility |
| Mode    | Most frequent value            | Categorical data     | Works with any data type | May not exist/be unique   |

**Python Implementation:**

```
import numpy as np
import pandas as pd
from scipy import stats
```

```

# Sample data
data = [1, 2, 2, 3, 4, 100] # Contains outlier

# Calculate measures
mean_val = np.mean(data)      # 18.67 (affected by outlier)
median_val = np.median(data)  # 2.5 (robust to outlier)
mode_val = stats.mode(data)[0] # 2

print(f"Mean: {mean_val}")
print(f"Median: {median_val}")
print(f"Mode: {mode_val}")

# When to use each
def choose_central_tendency(data):
    skewness = stats.skew(data)

    if abs(skewness) < 0.5:
        return "Mean (approximately symmetric)"
    elif skewness > 0.5:
        return "Median (right-skewed)"
    else:
        return "Median (left-skewed)"

```

## Q2: Explain measures of variability and their interpretations.

**Answer:**

### Measures of Variability:

#### 1. Range:

- **Formula:**  $Range = Max - Min$
- **Use:** Quick measure of spread
- **Limitation:** Sensitive to outliers

#### 2. Variance:

- **Population:**  $\sigma^2 = \frac{\sum (x_i - \mu)^2}{N}$
- **Sample:**  $s^2 = \frac{\sum (x_i - \bar{x})^2}{n-1}$

#### 3. Standard Deviation:

- **Population:**  $\sigma = \sqrt{\sigma^2}$
- **Sample:**  $s = \sqrt{s^2}$

#### 4. Interquartile Range (IQR):

- **Formula:**  $IQR = Q3 - Q1$
- **Use:** Robust measure of spread

### Python Implementation:

```

import numpy as np
import matplotlib.pyplot as plt

def analyze_variability(data, data_name="Dataset"):
    """Comprehensive variability analysis"""

    # Calculate measures
    range_val = np.max(data) - np.min(data)
    variance = np.var(data, ddof=1) # Sample variance
    std_dev = np.std(data, ddof=1) # Sample standard deviation
    iqr = np.percentile(data, 75) - np.percentile(data, 25)

    # Coefficient of variation
    cv = (std_dev / np.mean(data)) * 100

    results = {
        'Range': range_val,
        'Variance': variance,
        'Standard Deviation': std_dev,
        'IQR': iqr,
        'Coefficient of Variation (%)': cv
    }

    print(f"\n{data_name} Variability Analysis:")
    print("-" * 40)
    for measure, value in results.items():
        print(f"{measure}: {value:.3f}")

    # Interpretation
    if cv < 15:
        variability = "Low"
    elif cv < 35:
        variability = "Moderate"
    else:
        variability = "High"

    print(f"\nOverall Variability: {variability}")

    return results

# Example usage
dataset1 = np.random.normal(100, 15, 1000) # Mean=100, SD=15
dataset2 = np.random.exponential(2, 1000) # Exponential distribution

analyze_variability(dataset1, "Normal Distribution")
analyze_variability(dataset2, "Exponential Distribution")

```

### Q3: What is the difference between population and sample statistics?

**Answer:**

**Population vs Sample Statistics:**

| Aspect                    | Population                 | Sample               |
|---------------------------|----------------------------|----------------------|
| <b>Definition</b>         | Entire group of interest   | Subset of population |
| <b>Size</b>               | N (usually large/infinite) | n (smaller than N)   |
| <b>Mean Symbol</b>        | $\mu$ (mu)                 | $\bar{x}$ (x-bar)    |
| <b>Variance Symbol</b>    | $\sigma^2$ (sigma squared) | $s^2$                |
| <b>Standard Deviation</b> | $\sigma$ (sigma)           | s                    |
| <b>Purpose</b>            | True parameter (unknown)   | Estimate parameter   |

## Why Sample Statistics Matter:

```
import numpy as np
import matplotlib.pyplot as plt

# Simulation: Population vs Sample Statistics
np.random.seed(42)

# True population (infinite normal distribution)
population_mean = 50
population_std = 10

# Generate samples of different sizes
sample_sizes = [10, 30, 100, 500, 1000]
num_samples = 1000

results = {}

for n in sample_sizes:
    sample_means = []
    sample_stds = []

    # Generate many samples of size n
    for _ in range(num_samples):
        sample = np.random.normal(population_mean, population_std, n)
        sample_means.append(np.mean(sample))
        sample_stds.append(np.std(sample, ddof=1)) # Sample std

    results[n] = {
        'mean_of_sample_means': np.mean(sample_means),
        'std_of_sample_means': np.std(sample_means),
        'mean_of_sample_stds': np.mean(sample_stds)
    }

# Display results
print("Sample Size Analysis:")
print("-" * 60)
print(f"{'Sample Size':<12} {'Mean of Means':<15} {'Std of Means':<15} {'Mean of Sample Means':<15} {'Mean of Sample Stds':<15}")
print("-" * 60)

for n, stats in results.items():
    print(f"{n:<12} {stats['mean_of_sample_means']:<15.3f} "
          f"{stats['std_of_sample_means']:<15.3f} {stats['mean_of_sample_stds']:<15.3f} ")
```

```
print(f"\nTrue Population Mean: {population_mean}")
print(f"True Population Std: {population_std}")
```

## Probability Theory

### Q4: Explain the fundamental rules of probability.

**Answer:**

#### Basic Probability Rules:

##### 1. Addition Rule:

- **Mutually Exclusive:**  $P(A \cup B) = P(A) + P(B)$
- **General Case:**  $P(A \cup B) = P(A) + P(B) - P(A \cap B)$

##### 2. Multiplication Rule:

- **Independent Events:**  $P(A \cap B) = P(A) \times P(B)$
- **Dependent Events:**  $P(A \cap B) = P(A) \times P(B|A)$

##### 3. Conditional Probability:

- $P(A|B) = \frac{P(A \cap B)}{P(B)}$  where

##### 4. Law of Total Probability:

- $P(A) = \sum_i P(A|B_i) \times P(B_i)$

#### Python Implementation:

```
import numpy as np
from itertools import combinations

class ProbabilityCalculator:
    def __init__(self):
        pass

    def addition_rule(self, p_a, p_b, p_a_and_b=0, exclusive=False):
        """Calculate P(A or B)"""
        if exclusive:
            return p_a + p_b
        else:
            return p_a + p_b - p_a_and_b

    def multiplication_rule(self, p_a, p_b_given_a=None, independent=True):
        """Calculate P(A and B)"""
        if independent:
            return p_a * p_b_given_a # p_b_given_a is actually just P(B)
        else:
            return p_a * p_b_given_a
```

```

def conditional_probability(self, p_a_and_b, p_b):
    """Calculate P(A|B)"""
    if p_b == 0:
        return "Undefined (P(B) = 0)"
    return p_a_and_b / p_b

def bayes_theorem(self, p_a, p_b_given_a, p_b_given_not_a):
    """Calculate P(A|B) using Bayes' theorem"""
    p_not_a = 1 - p_a
    p_b = p_b_given_a * p_a + p_b_given_not_a * p_not_a

    if p_b == 0:
        return "Undefined"

    return (p_b_given_a * p_a) / p_b

# Example: Medical Diagnosis Problem
def medical_diagnosis_example():
    """
    Example: Disease diagnosis with test accuracy
    - Disease prevalence: 1%
    - Test sensitivity (true positive rate): 95%
    - Test specificity (true negative rate): 90%

    What's P(Disease | Positive Test)?
    """

    calc = ProbabilityCalculator()

    # Given probabilities
    p_disease = 0.01 # Prior probability
    p_positive_given_disease = 0.95 # Sensitivity
    p_negative_given_no_disease = 0.90 # Specificity
    p_positive_given_no_disease = 1 - p_negative_given_no_disease # False positive rate

    # Calculate P(Disease | Positive Test)
    p_disease_given_positive = calc.bayes_theorem(
        p_disease, p_positive_given_disease, p_positive_given_no_disease
    )

    print("Medical Diagnosis Example:")
    print(f"Prior probability of disease: {p_disease:.1%}")
    print(f"Test sensitivity: {p_positive_given_disease:.1%}")
    print(f"Test specificity: {p_negative_given_no_disease:.1%}")
    print(f"Posterior probability (Disease | Positive): {p_disease_given_positive:.1%}")

    return p_disease_given_positive

medical_diagnosis_example()

```

## Q5: What is Bayes' Theorem and its applications?

Answer:

Bayes' Theorem:

$$P(A|B) = \frac{P(B|A) \times P(A)}{P(B)}$$

Components:

- **P(A|B)**: Posterior probability
- **P(B|A)**: Likelihood
- **P(A)**: Prior probability
- **P(B)**: Marginal probability (evidence)

Applications:

1. **Medical Diagnosis**
2. **Spam Detection**
3. **Machine Learning (Naive Bayes)**
4. **A/B Testing**
5. **Risk Assessment**

Comprehensive Example:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

class BayesianAnalysis:
    def __init__(self):
        self.prior = None
        self.posterior = None

    def spam_detection_example(self):
        """Naive Bayes for spam detection"""

        # Training data
        spam_emails = 1000
        ham_emails = 4000
        total_emails = spam_emails + ham_emails

        # Word "free" occurrences
        free_in_spam = 800
        free_in_ham = 200

        # Calculate probabilities
        p_spam = spam_emails / total_emails
        p_ham = ham_emails / total_emails
        p_free_given_spam = free_in_spam / spam_emails
```

```

p_free_given_ham = free_in_ham / ham_emails

# P(free) - total probability
p_free = p_free_given_spam * p_spam + p_free_given_ham * p_ham

# Bayes' theorem: P(spam | "free")
p_spam_given_free = (p_free_given_spam * p_spam) / p_free

print("Spam Detection Example:")
print(f"Prior P(spam): {p_spam:.3f}")
print(f"P('free' | spam): {p_free_given_spam:.3f}")
print(f"P('free' | ham): {p_free_given_ham:.3f}")
print(f"Posterior P(spam | 'free'): {p_spam_given_free:.3f}")

return p_spam_given_free

def ab_test_bayesian(self, control_conversions, control_visitors,
                    treatment_conversions, treatment_visitors):
    """Bayesian A/B testing"""

    # Beta prior (uninformative)
    alpha_prior = 1
    beta_prior = 1

    # Posterior parameters
    alpha_control = alpha_prior + control_conversions
    beta_control = beta_prior + control_visitors - control_conversions

    alpha_treatment = alpha_prior + treatment_conversions
    beta_treatment = beta_prior + treatment_visitors - treatment_conversions

    # Generate posterior samples
    n_samples = 10000
    control_samples = np.random.beta(alpha_control, beta_control, n_samples)
    treatment_samples = np.random.beta(alpha_treatment, beta_treatment, n_samples)

    # Calculate probability that treatment > control
    prob_treatment_better = np.mean(treatment_samples > control_samples)

    # Credible intervals
    control_ci = np.percentile(control_samples, [2.5, 97.5])
    treatment_ci = np.percentile(treatment_samples, [2.5, 97.5])

    print("Bayesian A/B Test Results:")
    print(f"Control conversion rate: {control_conversions/control_visitors:.3f}")
    print(f"Treatment conversion rate: {treatment_conversions/treatment_visitors:.3f}")
    print(f"P(Treatment > Control): {prob_treatment_better:.3f}")
    print(f"Control 95% CI: [{control_ci[0]:.3f}, {control_ci[1]:.3f}]")
    print(f"Treatment 95% CI: [{treatment_ci[0]:.3f}, {treatment_ci[1]:.3f}]")

    return prob_treatment_better

def conjugate_prior_example(self):
    """Beta-Binomial conjugate prior example"""

    # Prior belief: Beta(2, 8) - skeptical about high conversion rates

```



```

alpha_prior = 2
beta_prior = 8

# Observed data
successes = [5, 12, 18, 25] # Cumulative successes
trials = [10, 20, 30, 40]    # Cumulative trials

fig, axes = plt.subplots(2, 2, figsize=(12, 10))
axes = axes.ravel()

x = np.linspace(0, 1, 1000)

for i, (s, n) in enumerate(zip(successes, trials)):
    # Update posterior
    alpha_post = alpha_prior + s
    beta_post = beta_prior + n - s

    # Plot prior and posterior
    prior = stats.beta(alpha_prior, beta_prior)
    posterior = stats.beta(alpha_post, beta_post)

    axes[i].plot(x, prior.pdf(x), 'b--', label='Prior', alpha=0.7)
    axes[i].plot(x, posterior.pdf(x), 'r-', label='Posterior')
    axes[i].set_title(f'After {n} trials, {s} successes')
    axes[i].legend()
    axes[i].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

# Run examples
bayes = BayesianAnalysis()
bayes.spam_detection_example()
print("\n" + "="*50 + "\n")
bayes.ab_test_bayesian(250, 5000, 310, 5200) # Example A/B test data

```

## Probability Distributions

**Q6: Compare and contrast the most important probability distributions.**

**Answer:**

**Key Probability Distributions:**

| Distribution       | Type       | Parameters      | Mean        | Variance      | Use Cases              |
|--------------------|------------|-----------------|-------------|---------------|------------------------|
| <b>Normal</b>      | Continuous | $\mu, \sigma^2$ | $\mu$       | $\sigma^2$    | Natural phenomena, CLT |
| <b>Binomial</b>    | Discrete   | $n, p$          | $np$        | $np(1-p)$     | Success/failure trials |
| <b>Poisson</b>     | Discrete   | $\lambda$       | $\lambda$   | $\lambda$     | Rare events, counts    |
| <b>Exponential</b> | Continuous | $\lambda$       | $1/\lambda$ | $1/\lambda^2$ | Time between events    |

| Distribution | Type       | Parameters      | Mean                    | Variance                                         | Use Cases             |
|--------------|------------|-----------------|-------------------------|--------------------------------------------------|-----------------------|
| Beta         | Continuous | $\alpha, \beta$ | $\alpha/(\alpha+\beta)$ | $\alpha\beta/[(\alpha+\beta)^2(\alpha+\beta+1)]$ | Proportions, Bayesian |

Detailed Analysis:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats
import seaborn as sns

class DistributionAnalyzer:
    def __init__(self):
        self.distributions = {}

    def plot_distributions_comparison(self):
        """Compare major probability distributions"""

        fig, axes = plt.subplots(3, 2, figsize=(15, 12))

        # Normal Distribution
        x_norm = np.linspace(-4, 4, 1000)
        for mu, sigma in [(0, 1), (0, 0.5), (1, 1)]:
            y = stats.norm.pdf(x_norm, mu, sigma)
            axes[0,0].plot(x_norm, y, label=f'μ={mu}, σ={sigma}')
        axes[0,0].set_title('Normal Distribution')
        axes[0,0].legend()
        axes[0,0].grid(True, alpha=0.3)

        # Binomial Distribution
        x_binom = np.arange(0, 21)
        for n, p in [(20, 0.3), (20, 0.5), (20, 0.7)]:
            y = stats.binom.pmf(x_binom, n, p)
            axes[0,1].plot(x_binom, y, 'o-', label=f'n={n}, p={p}')
        axes[0,1].set_title('Binomial Distribution')
        axes[0,1].legend()
        axes[0,1].grid(True, alpha=0.3)

        # Poisson Distribution
        x_poisson = np.arange(0, 20)
        for lam in [1, 3, 7]:
            y = stats.poisson.pmf(x_poisson, lam)
            axes[1,0].plot(x_poisson, y, 'o-', label=f'λ={lam}')
        axes[1,0].set_title('Poisson Distribution')
        axes[1,0].legend()
        axes[1,0].grid(True, alpha=0.3)

        # Exponential Distribution
        x_exp = np.linspace(0, 5, 1000)
        for lam in [0.5, 1, 2]:
            y = stats.expon.pdf(x_exp, scale=1/lam)
            axes[1,1].plot(x_exp, y, label=f'λ={lam}')
        axes[1,1].set_title('Exponential Distribution')
        axes[1,1].legend()
        axes[1,1].grid(True, alpha=0.3)
```

```

# Beta Distribution
x_beta = np.linspace(0, 1, 1000)
for alpha, beta in [(2, 2), (5, 2), (2, 5)]:
    y = stats.beta.pdf(x_beta, alpha, beta)
    axes[2,0].plot(x_beta, y, label=f' $\alpha={\alpha}$ ,  $\beta={\beta}$ ')
axes[2,0].set_title('Beta Distribution')
axes[2,0].legend()
axes[2,0].grid(True, alpha=0.3)

# Chi-Square Distribution
x_chi2 = np.linspace(0, 15, 1000)
for df in [1, 3, 7]:
    y = stats.chi2.pdf(x_chi2, df)
    axes[2,1].plot(x_chi2, y, label=f'df={df}')
axes[2,1].set_title('Chi-Square Distribution')
axes[2,1].legend()
axes[2,1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

def distribution_properties_test(self):
    """Test theoretical vs empirical properties"""

    sample_size = 10000

    distributions = {
        'Normal(0,1)': stats.norm(0, 1),
        'Binomial(20,0.3)': stats.binom(20, 0.3),
        'Poisson(3)': stats.poisson(3),
        'Exponential(2)': stats.expon(scale=0.5),
        'Beta(2,5)': stats.beta(2, 5)
    }

    results = []

    for name, dist in distributions.items():
        # Generate samples
        samples = dist.rvs(sample_size)

        # Calculate properties
        theoretical_mean = dist.mean()
        theoretical_var = dist.var()
        empirical_mean = np.mean(samples)
        empirical_var = np.var(samples, ddof=1)

        results.append({
            'Distribution': name,
            'Theoretical Mean': theoretical_mean,
            'Empirical Mean': empirical_mean,
            'Theoretical Var': theoretical_var,
            'Empirical Var': empirical_var,
            'Mean Error': abs(theoretical_mean - empirical_mean),
            'Var Error': abs(theoretical_var - empirical_var)
        })

```

```

import pandas as pd
df = pd.DataFrame(results)
print("Theoretical vs Empirical Properties:")
print(df.round(4))

return df

# Run analysis
analyzer = DistributionAnalyzer()
analyzer.plot_distributions_comparison()
analyzer.distribution_properties_test()

```

## Q7: When and why do we use the t-distribution instead of the normal distribution?

**Answer:**

### t-Distribution vs Normal Distribution:

#### Key Differences:

- **t-distribution:** Heavier tails, more variability
- **Normal distribution:** Thinner tails, less variability
- **Degrees of freedom:** t-distribution parameter affecting shape

#### When to Use t-Distribution:

1. **Small sample sizes** (typically  $n < 30$ )
2. **Unknown population variance**
3. **Population is approximately normal**
4. **Confidence intervals and hypothesis tests**

#### Mathematical Properties:

```

import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

class TDistributionAnalysis:
    def __init__(self):
        pass

    def compare_t_normal(self):
        """Compare t-distribution with normal distribution"""

        x = np.linspace(-4, 4, 1000)

        fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 6))

        # Plot t-distributions with different df
        normal = stats.norm(0, 1)
        ax1.plot(x, normal.pdf(x), 'b-', linewidth=2, label='Normal(0,1)')

```

```

for df in [1, 3, 10, 30]:
    t_dist = stats.t(df)
    ax1.plot(x, t_dist.pdf(x), '--', label=f't(df={df})')

ax1.set_title('t-Distribution vs Normal Distribution')
ax1.legend()
ax1.grid(True, alpha=0.3)

# Compare critical values
confidence_levels = [0.90, 0.95, 0.99]
sample_sizes = [5, 10, 15, 20, 30, 50, 100]

for conf_level in confidence_levels:
    alpha = 1 - conf_level

    normal_critical = stats.norm.ppf(1 - alpha/2)
    t_criticals = [stats.t.ppf(1 - alpha/2, n-1) for n in sample_sizes]

    ax2.plot(sample_sizes, t_criticals, 'o-',
              label=f'{conf_level:.0%} confidence')
    ax2.axhline(y=normal_critical, linestyle='--', alpha=0.7)

ax2.set_xlabel('Sample Size')
ax2.set_ylabel('Critical Value')
ax2.set_title('Critical Values: t vs Normal')
ax2.legend()
ax2.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

def practical_example(self):
    """Practical example: confidence intervals"""

    # Simulate small sample from normal population
    np.random.seed(42)
    population_mean = 100
    population_std = 15

    sample_sizes = [5, 10, 20, 50]
    confidence_level = 0.95
    alpha = 1 - confidence_level

    results = []

    for n in sample_sizes:
        sample = np.random.normal(population_mean, population_std, n)
        sample_mean = np.mean(sample)
        sample_std = np.std(sample, ddof=1)

        # Standard error
        se = sample_std / np.sqrt(n)

        # Critical values
        t_critical = stats.t.ppf(1 - alpha/2, n-1)

```

```

z_critical = stats.norm.ppf(1 - alpha/2)

# Confidence intervals
t_ci = (sample_mean - t_critical * se, sample_mean + t_critical * se)
z_ci = (sample_mean - z_critical * se, sample_mean + z_critical * se)

# CI widths
t_width = t_ci[1] - t_ci[0]
z_width = z_ci[1] - z_ci[0]

results.append({
    'Sample Size': n,
    'Sample Mean': sample_mean,
    't-CI': f'({t_ci[0]:.2f}, {t_ci[1]:.2f})',
    't-Width': t_width,
    'z-CI': f'({z_ci[0]:.2f}, {z_ci[1]:.2f})',
    'z-Width': z_width,
    'Width Difference': t_width - z_width
})

import pandas as pd
df = pd.DataFrame(results)
print("95% Confidence Intervals: t-distribution vs Normal")
print(f"True population mean: {population_mean}")
print("-" * 80)
print(df.to_string(index=False))

return df

# Run analysis
t_analysis = TDistributionAnalysis()
t_analysis.compare_t_normal()
t_analysis.practical_example()

```

## Sampling and Central Limit Theorem

**Q8: Explain the Central Limit Theorem and its implications.**

**Answer:**

**Central Limit Theorem (CLT):**

**Statement:** For large sample sizes ( $n \geq 30$ ), the sampling distribution of the sample mean approaches a normal distribution, regardless of the population distribution.

**Mathematical Form:**

$$\bar{X} \sim N\left(\mu, \frac{\sigma^2}{n}\right)$$

**Standardized Form:**

$$Z = \frac{\bar{X} - \mu}{\sigma/\sqrt{n}} \sim N(0, 1)$$

### Key Points:

1. **Sample size:** Generally  $n \geq 30$  is sufficient
2. **Any population distribution:** Works for uniform, skewed, bimodal, etc.
3. **Standard error:**  $SE = \frac{\sigma}{\sqrt{n}}$  decreases as  $n$  increases

### Comprehensive Demonstration:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats
import seaborn as sns

class CLTDemonstration:
    def __init__(self):
        self.sample_sizes = [1, 5, 10, 30, 100]
        self.num_samples = 1000

    def demonstrate_clt(self, population_dist='exponential'):
        """Demonstrate CLT with different population distributions"""

        fig, axes = plt.subplots(len(self.sample_sizes), 2, figsize=(15, 20))

        # Define population distributions
        if population_dist == 'exponential':
            population = lambda size: np.random.exponential(2, size)
            pop_mean, pop_std = 2, 2
            title_prefix = "Exponential"
        elif population_dist == 'uniform':
            population = lambda size: np.random.uniform(0, 10, size)
            pop_mean, pop_std = 5, np.sqrt(100/12)
            title_prefix = "Uniform"
        elif population_dist == 'bimodal':
            def bimodal_sample(size):
                choice = np.random.choice([0, 1], size, p=[0.3, 0.7])
                return np.where(choice, np.random.normal(2, 0.5, size),
                                np.random.normal(8, 1, size))
            population = bimodal_sample
            pop_mean, pop_std = 0.3*2 + 0.7*8, np.sqrt(0.3*(0.5**2 + 2**2) + 0.7*(1**2 +
            title_prefix = "Bimodal"

        for i, n in enumerate(self.sample_sizes):
            # Generate sample means
            sample_means = []
            for _ in range(self.num_samples):
                sample = population(n)
                sample_means.append(np.mean(sample))

            sample_means = np.array(sample_means)

            # Plot population distribution (for n=1, this is the sample distribution)
```

```

if n == 1:
    pop_sample = population(10000)
    axes[i, 0].hist(pop_sample, bins=50, density=True, alpha=0.7, color='blue')
    axes[i, 0].set_title(f'{title_prefix} Population Distribution')
else:
    axes[i, 0].hist(sample_means, bins=50, density=True, alpha=0.7, color='green')
    axes[i, 0].set_title(f'Sample Means Distribution (n={n})')

axes[i, 0].axvline(pop_mean, color='red', linestyle='--', label='Population Mean')
axes[i, 0].legend()
axes[i, 0].grid(True, alpha=0.3)

# Q-Q plot against normal distribution
theoretical_std = pop_std / np.sqrt(n)
standardized = (sample_means - pop_mean) / theoretical_std

stats.probplot(standardized, dist="norm", plot=axes[i, 1])
axes[i, 1].set_title(f'Q-Q Plot: Normal vs Sample Means (n={n})')
axes[i, 1].grid(True, alpha=0.3)

# Print statistics
empirical_mean = np.mean(sample_means)
empirical_std = np.std(sample_means, ddof=1)
theoretical_mean = pop_mean
theoretical_std = pop_std / np.sqrt(n)

print(f"Sample size n={n}:")
print(f"  Empirical mean: {empirical_mean:.3f}, Theoretical: {theoretical_mean:.3f}")
print(f"  Empirical std: {empirical_std:.3f}, Theoretical: {theoretical_std:.3f}")
print(f"  Difference in mean: {abs(empirical_mean - theoretical_mean):.3f}")
print(f"  Difference in std: {abs(empirical_std - theoretical_std):.3f}")
print()

plt.tight_layout()
plt.show()

def sampling_distribution_properties(self):
    """Analyze properties of sampling distributions"""

    # Different population distributions
    distributions = {
        'Normal': lambda n: np.random.normal(50, 10, n),
        'Exponential': lambda n: np.random.exponential(3, n),
        'Uniform': lambda n: np.random.uniform(0, 20, n),
        'Chi-square': lambda n: np.random.chisquare(5, n)
    }

    sample_sizes = [5, 15, 30, 50, 100]
    results = []

    for dist_name, dist_func in distributions.items():
        # Get population parameters
        large_sample = dist_func(100000)
        pop_mean = np.mean(large_sample)
        pop_std = np.std(large_sample, ddof=1)

```



```

for n in sample_sizes:
    sample_means = [np.mean(dist_func(n)) for _ in range(1000)]

    # Calculate properties
    empirical_mean = np.mean(sample_means)
    empirical_std = np.std(sample_means, ddof=1)
    theoretical_std = pop_std / np.sqrt(n)

    # Normality test (Shapiro-Wilk)
    if len(sample_means) <= 5000: # Shapiro-Wilk limit
        _, p_value = stats.shapiro(sample_means)
        is_normal = p_value > 0.05
    else:
        is_normal = "Not tested (sample too large)"

    results.append({
        'Distribution': dist_name,
        'Sample Size': n,
        'Pop Mean': pop_mean,
        'Emp Mean': empirical_mean,
        'Theo SE': theoretical_std,
        'Emp SE': empirical_std,
        'Normal?': is_normal
    })

import pandas as pd
df = pd.DataFrame(results)
print("Central Limit Theorem Verification:")
print(df.round(3))

return df

# Run demonstrations
clt = CLTDemonstration()

# Demonstrate with exponential distribution
print("CLT Demonstration with Exponential Distribution:")
clt.demonstrate_clt('exponential')

# Analyze properties
clt.sampling_distribution_properties()

```

## Q9: What are different sampling methods and their trade-offs?

**Answer:**

### Sampling Methods Comparison:

| Method               | Description                    | Advantages             | Disadvantages      | When to Use               |
|----------------------|--------------------------------|------------------------|--------------------|---------------------------|
| <b>Simple Random</b> | Every element has equal chance | Unbiased, easy         | May miss subgroups | Homogeneous populations   |
| <b>Stratified</b>    | Population divided into strata | Ensures representation | Complex design     | Heterogeneous populations |

| Method             | Description                | Advantages        | Disadvantages         | When to Use          |
|--------------------|----------------------------|-------------------|-----------------------|----------------------|
| <b>Systematic</b>  | Every kth element          | Easy to implement | Periodicity issues    | Large, ordered lists |
| <b>Cluster</b>     | Sample entire clusters     | Cost-effective    | Higher sampling error | Geographic studies   |
| <b>Convenience</b> | Easily accessible subjects | Quick and cheap   | High bias potential   | Exploratory research |

## Implementation and Analysis:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy import stats

class SamplingMethodsAnalysis:
    def __init__(self):
        # Create a synthetic population
        np.random.seed(42)
        self.population_size = 10000
        self.create_population()

    def create_population(self):
        """Create a heterogeneous population with different groups"""

        # Three groups with different characteristics
        group1_size = 3000 # High income, high education
        group2_size = 4000 # Medium income, medium education
        group3_size = 3000 # Low income, low education

        # Group 1: High socioeconomic status
        income1 = np.random.normal(80000, 15000, group1_size)
        education1 = np.random.normal(16, 2, group1_size)
        satisfaction1 = np.random.normal(7.5, 1.2, group1_size)
        group1 = np.ones(group1_size) * 1

        # Group 2: Medium socioeconomic status
        income2 = np.random.normal(50000, 12000, group2_size)
        education2 = np.random.normal(13, 2.5, group2_size)
        satisfaction2 = np.random.normal(6.0, 1.5, group2_size)
        group2 = np.ones(group2_size) * 2

        # Group 3: Low socioeconomic status
        income3 = np.random.normal(25000, 8000, group3_size)
        education3 = np.random.normal(10, 2, group3_size)
        satisfaction3 = np.random.normal(4.5, 1.8, group3_size)
        group3 = np.ones(group3_size) * 3

        # Combine all groups
        self.population = pd.DataFrame({
            'income': np.concatenate([income1, income2, income3]),
            'education': np.concatenate([education1, education2, education3]),
            'satisfaction': np.concatenate([satisfaction1, satisfaction2, satisfaction3]),
            'group': np.concatenate([group1, group2, group3])
        })
```

```

    })

    # Add geographic clustering (for cluster sampling)
    n_clusters = 100
    cluster_size = self.population_size // n_clusters
    self.population['cluster'] = np.repeat(range(n_clusters), cluster_size)

    print("Population created:")
    print(self.population.groupby('group').agg({
        'income': 'mean',
        'education': 'mean',
        'satisfaction': 'mean'
    }).round(1))

def simple_random_sampling(self, sample_size=500):
    """Simple random sampling"""
    sample = self.population.sample(n=sample_size, random_state=42)
    return sample

def stratified_sampling(self, sample_size=500):
    """Stratified sampling proportional to group sizes"""
    strata_props = self.population['group'].value_counts(normalize=True).sort_index()

    stratified_sample = pd.DataFrame()
    for group, prop in strata_props.items():
        group_sample_size = int(sample_size * prop)
        group_data = self.population[self.population['group'] == group]
        group_sample = group_data.sample(n=group_sample_size, random_state=42)
        stratified_sample = pd.concat([stratified_sample, group_sample])

    return stratified_sample

def systematic_sampling(self, sample_size=500):
    """Systematic sampling"""
    k = len(self.population) // sample_size
    start = np.random.randint(0, k)
    indices = range(start, len(self.population), k)[:sample_size]

    return self.population.iloc[indices]

def cluster_sampling(self, n_clusters_sample=20):
    """Cluster sampling - select entire clusters"""
    total_clusters = self.population['cluster'].nunique()
    selected_clusters = np.random.choice(
        range(total_clusters), size=n_clusters_sample, replace=False
    )

    cluster_sample = self.population[
        self.population['cluster'].isin(selected_clusters)
    ]

    return cluster_sample

def compare_sampling_methods(self, n_simulations=100):
    """Compare bias and variance of different sampling methods"""

```

```

# True population parameters
true_income = self.population['income'].mean()
true_education = self.population['education'].mean()
true_satisfaction = self.population['satisfaction'].mean()

results = {method: {'income': [], 'education': [], 'satisfaction': []}
           for method in ['Simple Random', 'Stratified', 'Systematic', 'Cluster']}

for i in range(n_simulations):
    # Simple random sampling
    srs_sample = self.simple_random_sampling()
    results['Simple Random']['income'].append(srs_sample['income'].mean())
    results['Simple Random']['education'].append(srs_sample['education'].mean())
    results['Simple Random']['satisfaction'].append(srs_sample['satisfaction'].mean())

    # Stratified sampling
    strat_sample = self.stratified_sampling()
    results['Stratified']['income'].append(strat_sample['income'].mean())
    results['Stratified']['education'].append(strat_sample['education'].mean())
    results['Stratified']['satisfaction'].append(strat_sample['satisfaction'].mean())

    # Systematic sampling
    sys_sample = self.systematic_sampling()
    results['Systematic']['income'].append(sys_sample['income'].mean())
    results['Systematic']['education'].append(sys_sample['education'].mean())
    results['Systematic']['satisfaction'].append(sys_sample['satisfaction'].mean())

    # Cluster sampling
    cluster_sample = self.cluster_sampling()
    results['Cluster']['income'].append(cluster_sample['income'].mean())
    results['Cluster']['education'].append(cluster_sample['education'].mean())
    results['Cluster']['satisfaction'].append(cluster_sample['satisfaction'].mean())

# Analyze results
comparison_results = []
true_values = {'income': true_income, 'education': true_education,
               'satisfaction': true_satisfaction}

for method in results:
    for variable in ['income', 'education', 'satisfaction']:
        estimates = results[method][variable]

        bias = np.mean(estimates) - true_values[variable]
        variance = np.var(estimates, ddof=1)
        mse = bias**2 + variance

        comparison_results.append({
            'Method': method,
            'Variable': variable,
            'True Value': true_values[variable],
            'Mean Estimate': np.mean(estimates),
            'Bias': bias,
            'Variance': variance,
            'MSE': mse,
            'Std Error': np.sqrt(variance)
        })

```

```

comparison_df = pd.DataFrame(comparison_results)

# Create visualization
fig, axes = plt.subplots(1, 3, figsize=(18, 6))
variables = ['income', 'education', 'satisfaction']

for i, var in enumerate(variables):
    var_data = comparison_df[comparison_df['Variable'] == var]

    methods = var_data['Method']
    biases = var_data['Bias']
    std_errors = var_data['Std Error']

    x_pos = np.arange(len(methods))

    # Plot bias
    bars1 = axes[i].bar(x_pos - 0.2, biases, 0.4, label='Bias', alpha=0.7)

    # Plot standard error
    bars2 = axes[i].bar(x_pos + 0.2, std_errors, 0.4, label='Std Error', alpha=0.3)

    axes[i].set_xlabel('Sampling Method')
    axes[i].set_ylabel('Value')
    axes[i].set_title(f'{var.capitalize()} - Bias vs Standard Error')
    axes[i].set_xticks(x_pos)
    axes[i].set_xticklabels(methods, rotation=45)
    axes[i].legend()
    axes[i].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

print("Sampling Methods Comparison:")
print(comparison_df.round(2))

return comparison_df

# Run analysis
sampling_analysis = SamplingMethodsAnalysis()
comparison_results = sampling_analysis.compare_sampling_methods()

```

## Hypothesis Testing

**Q10: Explain the framework of hypothesis testing and common misconceptions.**

**Answer:**

**Hypothesis Testing Framework:**

**Step 1: State Hypotheses**

- **Null Hypothesis ( $H_0$ ):** Status quo, no effect

- **Alternative Hypothesis ( $H_1/H_a$ ):** What we want to prove

## Step 2: Choose Significance Level ( $\alpha$ )

- Common values: 0.05, 0.01, 0.10

## Step 3: Calculate Test Statistic

- z-test, t-test, chi-square, etc.

## Step 4: Determine p-value

- Probability of observing data given  $H_0$  is true

## Step 5: Make Decision

- If  $p\text{-value} \leq \alpha$ : Reject  $H_0$
- If  $p\text{-value} > \alpha$ : Fail to reject  $H_0$

## Common Misconceptions:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats
import seaborn as sns

class HypothesisTestingFramework:
    def __init__(self):
        self.alpha = 0.05

    def demonstrate_p_value_misconceptions(self):
        """Demonstrate common p-value misconceptions"""

        print("COMMON P-VALUE MISCONCEPTIONS:")
        print("=" * 50)

        # Misconception 1: p-value is probability that null hypothesis is true
        print("\n✗ MISCONCEPTION 1:")
        print("p-value = P( $H_0$  is true | data)")
        print("\n✓ CORRECT INTERPRETATION:")
        print("p-value = P(data or more extreme |  $H_0$  is true)")
        print("The p-value is NOT the probability that the null hypothesis is true!")

        # Misconception 2: 1 - p-value is probability that alternative is true
        print("\n✗ MISCONCEPTION 2:")
        print("P( $H_1$  is true) = 1 - p-value")
        print("\n✓ CORRECT:")
        print("p-value cannot directly tell us the probability of  $H_1$  being true.")
        print("This requires Bayesian analysis with prior probabilities.")

        # Misconception 3: Failing to reject means accepting null
        print("\n✗ MISCONCEPTION 3:")
        print("p > 0.05 means we 'accept' the null hypothesis")
        print("\n✓ CORRECT:")
        print("p > 0.05 means we 'fail to reject' the null hypothesis.")
        print("Absence of evidence is not evidence of absence!")
```

```

self.simulate_p_value_distribution()

def simulate_p_value_distribution(self):
    """Simulate p-value distribution under null and alternative"""

    n_simulations = 10000
    sample_size = 30

    # Scenario 1: Null is true ( $\mu = 0$ )
    p_values_null = []
    for _ in range(n_simulations):
        sample = np.random.normal(0, 1, sample_size) # True mean = 0
        t_stat, p_val = stats.ttest_1samp(sample, 0)
        p_values_null.append(p_val)

    # Scenario 2: Alternative is true ( $\mu = 0.5$ )
    p_values_alt = []
    for _ in range(n_simulations):
        sample = np.random.normal(0.5, 1, sample_size) # True mean = 0.5
        t_stat, p_val = stats.ttest_1samp(sample, 0) # Test against  $H_0: \mu = 0$ 
        p_values_alt.append(p_val)

    # Plot distributions
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 6))

    # Under null hypothesis
    ax1.hist(p_values_null, bins=50, density=True, alpha=0.7,
            label='p-values under  $H_0$ ')
    ax1.axhline(y=1, color='red', linestyle='--',
            label='Uniform distribution (y=1)')
    ax1.axvline(x=0.05, color='orange', linestyle='--',
            label=' $\alpha = 0.05$ ')
    ax1.set_xlabel('p-value')
    ax1.set_ylabel('Density')
    ax1.set_title('p-value Distribution When  $H_0$  is True')
    ax1.legend()
    ax1.grid(True, alpha=0.3)

    # Under alternative hypothesis
    ax2.hist(p_values_alt, bins=50, density=True, alpha=0.7,
            label='p-values under  $H_1$ ', color='green')
    ax2.axvline(x=0.05, color='orange', linestyle='--',
            label=' $\alpha = 0.05$ ')
    ax2.set_xlabel('p-value')
    ax2.set_ylabel('Density')
    ax2.set_title('p-value Distribution When  $H_1$  is True (Effect = 0.5)')
    ax2.legend()
    ax2.grid(True, alpha=0.3)

    plt.tight_layout()
    plt.show()

    # Calculate Type I and Type II error rates
    type_i_error = np.mean(np.array(p_values_null) <= 0.05)
    type_ii_error = np.mean(np.array(p_values_alt) > 0.05)
    power = 1 - type_ii_error

```

```

print(f"\nSimulation Results:")
print(f"Type I Error Rate (should  $\approx$  0.05): {type_i_error:.3f}")
print(f"Type II Error Rate: {type_ii_error:.3f}")
print(f"Statistical Power: {power:.3f}")

def power_analysis(self):
    """Comprehensive power analysis"""

    # Parameters
    sample_sizes = range(5, 101, 5)
    effect_sizes = [0.2, 0.5, 0.8] # Small, medium, large
    alpha = 0.05

    fig, ax = plt.subplots(figsize=(12, 8))

    for effect_size in effect_sizes:
        powers = []

        for n in sample_sizes:
            # Calculate power for one-sample t-test
            # Power = P(reject H0 | H1 is true)

            # Critical value under H0
            t_critical = stats.t.ppf(1 - alpha/2, n-1)

            # Non-centrality parameter
            delta = effect_size * np.sqrt(n)

            # Power using non-central t-distribution
            power = 1 - stats.nct.cdf(t_critical, n-1, delta) + \
                stats.nct.cdf(-t_critical, n-1, delta)

            powers.append(power)

        ax.plot(sample_sizes, powers, 'o-',
                label=f'Effect size = {effect_size}', linewidth=2)

    ax.axhline(y=0.8, color='red', linestyle='--',
                label='80% Power threshold')
    ax.set_xlabel('Sample Size')
    ax.set_ylabel('Statistical Power')
    ax.set_title('Power Analysis: Effect Size vs Sample Size')
    ax.legend()
    ax.grid(True, alpha=0.3)
    ax.set_ylim(0, 1)

    plt.show()

    return powers

# Multiple testing problem
class MultipleTestingProblem:
    def __init__(self):
        pass

```



```

def demonstrate_multiple_testing(self):
    """Demonstrate the multiple testing problem"""

    n_tests = [1, 5, 10, 20, 50, 100]
    alpha = 0.05

    # Calculate family-wise error rate
    fwer = [1 - (1 - alpha)**m for m in n_tests]

    # Bonferroni correction
    bonferroni_alpha = [alpha / m for m in n_tests]

    # Simulation
    n_simulations = 10000
    simulation_results = []

    for m in n_tests:
        false_positives = 0

        for _ in range(n_simulations):
            # Generate m independent tests under null
            p_values = np.random.uniform(0, 1, m) # Uniform under null

            # Count false positives
            if np.any(p_values <= alpha):
                false_positives += 1

        simulation_results.append(false_positives / n_simulations)

    # Plot results
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 6))

    # Family-wise error rate
    ax1.plot(n_tests, fwer, 'ro-', label='Theoretical FWER')
    ax1.plot(n_tests, simulation_results, 'bs-', label='Simulated FWER')
    ax1.axhline(y=alpha, color='green', linestyle='--',
                label=f' $\alpha = \{alpha\}$ ')
    ax1.set_xlabel('Number of Tests')
    ax1.set_ylabel('Family-wise Error Rate')
    ax1.set_title('Multiple Testing Problem')
    ax1.legend()
    ax1.grid(True, alpha=0.3)

    # Bonferroni correction
    ax2.semilogy(n_tests, bonferroni_alpha, 'go-',
                 label='Bonferroni-corrected  $\alpha$ ')
    ax2.axhline(y=alpha, color='red', linestyle='--',
                label=f'Original  $\alpha = \{alpha\}$ ')
    ax2.set_xlabel('Number of Tests')
    ax2.set_ylabel('Adjusted  $\alpha$  (log scale)')
    ax2.set_title('Bonferroni Correction')
    ax2.legend()
    ax2.grid(True, alpha=0.3)

    plt.tight_layout()
    plt.show()

```

```

print("Multiple Testing Problem Results:")
print("-" * 50)
for i, m in enumerate(n_tests):
    print(f"Tests: {m:3d}, FWER: {fwer[i]:.3f}, "
          f"Simulated: {simulation_results[i]:.3f}, "
          f"Bonferroni  $\alpha$ : {bonferroni_alpha[i]:.6f}")

# Run demonstrations
ht_framework = HypothesisTestingFramework()
ht_framework.demonstrate_p_value_misconceptions()
ht_framework.power_analysis()

mt_problem = MultipleTestingProblem()
mt_problem.demonstrate_multiple_testing()

```

## Q11: What are Type I and Type II errors? How do you balance them?

**Answer:**

### Error Types in Hypothesis Testing:

| Reality vs Decision  | $H_0$ True                | $H_0$ False               |
|----------------------|---------------------------|---------------------------|
| Reject $H_0$         | Type I Error ( $\alpha$ ) | Correct (Power)           |
| Fail to Reject $H_0$ | Correct ( $1-\alpha$ )    | Type II Error ( $\beta$ ) |

### Key Relationships:

- **Type I Error ( $\alpha$ ):** False positive rate
- **Type II Error ( $\beta$ ):** False negative rate
- **Power ( $1-\beta$ ):** True positive rate
- **Specificity ( $1-\alpha$ ):** True negative rate

### Comprehensive Analysis:

```

import numpy as np
import matplotlib.pyplot as plt
from scipy import stats
import pandas as pd

class ErrorAnalysis:
    def __init__(self):
        self.alpha = 0.05

    def visualize_errors(self):
        """Visual representation of Type I and Type II errors"""

        # Set up the distributions
        x = np.linspace(-4, 6, 1000)

        #  $H_0: \mu = 0$ ,  $H_1: \mu = 2$ 

```

```

null_dist = stats.norm(0, 1)
alt_dist = stats.norm(2, 1)

# Critical value (one-tailed test)
critical_value = stats.norm.ppf(1 - self.alpha, 0, 1)

fig, (ax1, ax2, ax3) = plt.subplots(3, 1, figsize=(12, 12))

# Plot 1: Type I Error
ax1.plot(x, null_dist.pdf(x), 'b-', linewidth=2, label='H0:  $\mu = 0$ ')
ax1.axvline(critical_value, color='red', linestyle='--',
            label=f'Critical value = {critical_value:.2f}')

# Shade Type I error region
x_type1 = x[x >= critical_value]
ax1.fill_between(x_type1, null_dist.pdf(x_type1),
                 alpha=0.3, color='red', label=f'Type I Error = {self.alpha}')

ax1.set_title('Type I Error (False Positive)')
ax1.legend()
ax1.grid(True, alpha=0.3)

# Plot 2: Type II Error
ax2.plot(x, alt_dist.pdf(x), 'g-', linewidth=2, label='H1:  $\mu = 2$ ')
ax2.axvline(critical_value, color='red', linestyle='--',
            label=f'Critical value = {critical_value:.2f}')

# Calculate Type II error
beta = alt_dist.cdf(critical_value)
x_type2 = x[x <= critical_value]
ax2.fill_between(x_type2, alt_dist.pdf(x_type2),
                 alpha=0.3, color='orange', label=f'Type II Error ( $\beta$ ) = {beta:.3f}')

# Shade power region
x_power = x[x >= critical_value]
ax2.fill_between(x_power, alt_dist.pdf(x_power),
                 alpha=0.3, color='green', label=f'Power (1- $\beta$ ) = {1-beta:.3f}')

ax2.set_title('Type II Error (False Negative) and Power')
ax2.legend()
ax2.grid(True, alpha=0.3)

# Plot 3: Both distributions
ax3.plot(x, null_dist.pdf(x), 'b-', linewidth=2, label='H0:  $\mu = 0$ ')
ax3.plot(x, alt_dist.pdf(x), 'g-', linewidth=2, label='H1:  $\mu = 2$ ')
ax3.axvline(critical_value, color='red', linestyle='--',
            label='Decision boundary')

ax3.set_title('Complete Picture: Both Error Types')
ax3.legend()
ax3.grid(True, alpha=0.3)
ax3.set_xlabel('Test Statistic Value')

plt.tight_layout()
plt.show()

```

```

        return critical_value, beta

def tradeoff_analysis(self):
    """Analyze the tradeoff between Type I and Type II errors"""

    # Parameters
    null_mean = 0
    alt_mean = 1.5
    std_dev = 1
    sample_size = 25

    # Range of alpha values
    alphas = np.linspace(0.01, 0.20, 50)

    # Calculate corresponding betas and power
    betas = []
    powers = []

    for alpha in alphas:
        # Critical value for given alpha
        critical_value = stats.norm.ppf(1 - alpha, null_mean,
                                         std_dev/np.sqrt(sample_size))

        # Type II error rate
        beta = stats.norm.cdf(critical_value, alt_mean,
                              std_dev/np.sqrt(sample_size))
        betas.append(beta)
        powers.append(1 - beta)

    # Plot tradeoff
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 6))

    # Alpha vs Beta tradeoff
    ax1.plot(alphas, betas, 'r-', linewidth=2, label='Type II Error ( $\beta$ )')
    ax1.plot(alphas, alphas, 'b--', linewidth=2, label='Type I Error ( $\alpha$ )')
    ax1.set_xlabel('Type I Error Rate ( $\alpha$ )')
    ax1.set_ylabel('Error Rate')
    ax1.set_title('Type I vs Type II Error Tradeoff')
    ax1.legend()
    ax1.grid(True, alpha=0.3)

    # Alpha vs Power
    ax2.plot(alphas, powers, 'g-', linewidth=2)
    ax2.set_xlabel('Type I Error Rate ( $\alpha$ )')
    ax2.set_ylabel('Statistical Power ( $1-\beta$ )')
    ax2.set_title('Type I Error vs Statistical Power')
    ax2.grid(True, alpha=0.3)
    ax2.axhline(y=0.8, color='red', linestyle='--',
                label='80% Power threshold')
    ax2.legend()

    plt.tight_layout()
    plt.show()

    return alphas, betas, powers

```

```

def sample_size_calculation(self):
    """Calculate required sample size for desired power"""

    effect_sizes = [0.2, 0.5, 0.8, 1.0] # Cohen's d
    desired_power = 0.8
    alpha = 0.05

    results = []

    for effect_size in effect_sizes:
        # For two-sample t-test, approximate formula:
        #  $n \approx 2 * (z_{\alpha/2} + z_{\beta})^2 / d^2$ 

        z_alpha = stats.norm.ppf(1 - alpha/2) # Two-tailed
        z_beta = stats.norm.ppf(desired_power)

        n_per_group = 2 * (z_alpha + z_beta)**2 / (effect_size**2)

        # More accurate calculation using t-distribution
        # (iterative approach)
        n_accurate = self.calculate_sample_size_t_test(
            effect_size, alpha, desired_power
        )

        results.append({
            'Effect Size': effect_size,
            'Normal Approx': int(np.ceil(n_per_group)),
            'T-test Accurate': n_accurate,
            'Difference': n_accurate - int(np.ceil(n_per_group))
        })

    df = pd.DataFrame(results)
    print("Sample Size Requirements for 80% Power:")
    print(df)

    return df

def calculate_sample_size_t_test(self, effect_size, alpha, power,
                                max_n=1000):
    """Accurate sample size calculation for t-test"""

    for n in range(5, max_n):
        # Critical value
        t_critical = stats.t.ppf(1 - alpha/2, 2*n - 2)

        # Non-centrality parameter
        delta = effect_size * np.sqrt(n/2)

        # Calculate power
        calculated_power = 1 - stats.nct.cdf(t_critical, 2*n - 2, delta) + \
            stats.nct.cdf(-t_critical, 2*n - 2, delta)

        if calculated_power >= power:
            return n

    return max_n

```

```

# Cost-benefit analysis
class CostBenefitAnalysis:
    def __init__(self):
        pass

    def medical_test_example(self):
        """Medical diagnostic test cost-benefit analysis"""

        # Problem parameters
        disease_prevalence = 0.01 # 1% of population has disease
        test_sensitivity = 0.95    # P(positive | disease)
        test_specificity = 0.90    # P(negative | no disease)

        # Costs (in dollars)
        cost_test = 100
        cost_treatment_unnecessary = 5000 # False positive
        cost_missed_diagnosis = 50000    # False negative

        population_size = 100000

        # Calculate outcomes
        diseased = int(population_size * disease_prevalence)
        healthy = population_size - diseased

        # Test outcomes
        true_positives = int(diseased * test_sensitivity)
        false_negatives = diseased - true_positives

        true_negatives = int(healthy * test_specificity)
        false_positives = healthy - true_negatives

        # Costs
        total_test_cost = population_size * cost_test
        fp_cost = false_positives * cost_treatment_unnecessary
        fn_cost = false_negatives * cost_missed_diagnosis

        total_cost = total_test_cost + fp_cost + fn_cost

        print("Medical Test Cost-Benefit Analysis:")
        print("-" * 50)
        print(f"Population: {population_size:,}")
        print(f"Disease prevalence: {disease_prevalence:.1%}")
        print(f"Test sensitivity: {test_sensitivity:.1%}")
        print(f"Test specificity: {test_specificity:.1%}")
        print()
        print("Outcomes:")
        print(f"True positives: {true_positives:,}")
        print(f"False positives: {false_positives:,}")
        print(f"True negatives: {true_negatives:,}")
        print(f"False negatives: {false_negatives:,}")
        print()
        print("Costs:")
        print(f"Testing cost: ${total_test_cost:,}")
        print(f"False positive cost: ${fp_cost:,}")
        print(f"False negative cost: ${fn_cost:,}")

```

```

    print(f"Total cost: ${total_cost:,}")
    print()
    print(f"Cost per person: ${total_cost/population_size:.2f}")

    return {
        'total_cost': total_cost,
        'cost_per_person': total_cost/population_size,
        'false_positives': false_positives,
        'false_negatives': false_negatives
    }

# Run all analyses
error_analysis = ErrorAnalysis()

print("1. VISUALIZING TYPE I AND TYPE II ERRORS:")
critical_val, beta = error_analysis.visualize_errors()

print("\n2. TRADEOFF ANALYSIS:")
alphas, betas, powers = error_analysis.tradeoff_analysis()

print("\n3. SAMPLE SIZE CALCULATION:")
sample_size_df = error_analysis.sample_size_calculation()

print("\n4. COST-BENEFIT EXAMPLE:")
cost_benefit = CostBenefitAnalysis()
medical_results = cost_benefit.medical_test_example()

```

## Quick Reference Formulas

### Central Tendency & Variability

- **Mean:**  $\bar{x} = \frac{\sum x_i}{n}$
- **Variance:**  $s^2 = \frac{\sum (x_i - \bar{x})^2}{n-1}$
- **Standard Error:**  $SE = \frac{s}{\sqrt{n}}$

### Probability Rules

- **Addition:**  $P(A \cup B) = P(A) + P(B) - P(A \cap B)$
- **Multiplication:**  $P(A \cap B) = P(A) \times P(B|A)$
- **Bayes' Theorem:**  $P(A|B) = \frac{P(B|A) \times P(A)}{P(B)}$

### Hypothesis Testing

- **t-statistic:**  $t = \frac{\bar{x} - \mu_0}{s/\sqrt{n}}$
- **z-statistic:**  $z = \frac{\bar{x} - \mu_0}{\sigma/\sqrt{n}}$
- **Chi-square:**  $\chi^2 = \sum \frac{(O_i - E_i)^2}{E_i}$

## Confidence Intervals

- **Mean (t-dist):**  $\bar{x} \pm t_{\alpha/2, df} \times \frac{s}{\sqrt{n}}$
- **Proportion:**  $\hat{p} \pm z_{\alpha/2} \times \sqrt{\frac{\hat{p}(1-\hat{p})}{n}}$

## Interview Tips

1. **Always check assumptions** before applying tests
2. **Distinguish correlation from causation**
3. **Consider practical significance** vs statistical significance
4. **Know when to use parametric vs non-parametric tests**
5. **Understand p-values** - they are NOT error probabilities

*This comprehensive Statistics cheat sheet covers fundamental concepts through advanced hypothesis testing. Master these concepts with hands-on practice and real-world applications!*